Auditing Gender & Race Bias in Customer Service AI

A Responsible AI Research Project

End-to-End Bias Detection, Mitigation & Explainability

Technologies: Python | VADER | TextBlob | DistilBERT | Fairlearn | SHAP | LIME | Opacus

Dataset: 800 Controlled Customer Service Complaints

8 Demographic Groups | 4 Races | 2 Genders

Table of Contents

1. The Problem: Why This Project Exists
2. Our Approach: How We Prove Bias Exists
3. Step 1 - Dataset Generation (The Experiment Setup)
4. Step 2 - Sentiment Analysis (Running the Models)
5. Step 3 - Bias Detection (Statistical Proof)
6. Step 4 - Fairness Metrics (Measuring Inequality)
7. Step 5 - Bias Mitigation (Fixing the Bias)
8. Step 6 - Explainability (Why Does Bias Happen?)
9. Step 7 - Privacy (Protecting User Data)
10. Step 8 - Visualizations (Telling the Story)
11. The Demo App (Interactive Bias Detector)
12. Key Findings & Conclusion
13. Technical Architecture

1. The Problem: Why This Project Exists

What is happening in the real world?

Companies around the world use AI systems to handle customer complaints. When you call a helpline, write an email complaint, or use a chatbot, AI in the background is analyzing your message to decide:

- How urgent is this complaint?
- How angry is this customer?
- Should we escalate this to a manager?
- How quickly should we respond?

This AI process is called Sentiment Analysis - the AI reads your text and assigns a score from -1 (very negative) to +1 (very positive). A score of -0.8 means "this customer is very upset" and a score of -0.2 means "this customer is slightly unhappy."

Where is the problem?

The problem is: these AI systems don't just analyze the complaint - they also see the customer's NAME. And research has shown that AI models can treat the SAME complaint differently depending on whether the customer's name sounds like it belongs to a White person, Black person, Indian person, or Chinese person.

For example, imagine the exact same complaint:

"Brad is angry about the delayed delivery" -> Score: -0.55 (Negative)

"Jamal is angry about the delayed delivery" -> Score: -0.60 (More Negative)

Same words, same emotion, same complaint - but Jamal (a Black male name) gets a MORE NEGATIVE score than Brad (a White male name). This means Jamal's complaint would be flagged as more severe, potentially leading to different treatment in customer service queues.

Why does this matter?

This bias has real consequences:

- A Black customer might get escalated more aggressively, creating a negative impression of them
- Or worse - they might get LOWER priority because the system over-adjusts
- Indian or Chinese customers might face systematic scoring differences
- Companies could be unknowingly discriminating against customers based on race or gender

OUR GOAL: Scientifically prove whether this bias exists in popular AI sentiment analysis tools, measure how big the bias is, and show how to fix it.

2. Our Approach: How We Prove Bias Exists

We designed this project like a scientific experiment. Just like a medical trial tests whether a drug works by having a control group and a test group, we test whether AI is biased by changing only ONE thing: the customer's name.

The Experimental Design

Control Variable (kept same): The complaint text (exact same words)

Independent Variable (we change): The customer's name (signals race & gender)

Dependent Variable (we measure): The sentiment score the AI assigns

This is a classic controlled experiment. If the AI is truly fair, changing ONLY the name should NOT change the score at all. Any score difference must be caused by the name - which means the AI is biased.

The 8-Step Pipeline

Our project runs through 8 steps, each building on the previous one:

Step 1 - Dataset Generation: Create 800 controlled complaint sentences

Step 2 - Sentiment Analysis: Run all 800 sentences through 3 AI models

Step 3 - Bias Detection: Use statistics to prove if score differences are real

Step 4 - Fairness Metrics: Measure bias using 5 industry-standard fairness metrics

Step 5 - Bias Mitigation: Apply 3 techniques to fix/reduce the bias

Step 6 - Explainability: Use SHAP and LIME to explain WHY bias happens

Step 7 - Privacy: Ensure customer data is protected with differential privacy

Step 8 - Visualizations: Create 10 charts to tell the story visually

Think of it like a detective investigation: Step 1 sets the trap, Step 2 catches the suspect, Steps 3-4 gather evidence, Step 5 finds the fix, Step 6 explains the crime, Step 7 protects witnesses, and Step 8 presents the case to the jury.

3. Step 1 - Dataset Generation

What we created

We built a dataset of exactly 800 customer complaint sentences using a carefully controlled formula:

20 complaint templates x 8 demographic groups x 5 names per group = 800 sentences

The 20 Complaint Templates (4 categories)

We wrote 20 different complaint sentences covering 4 types of emotions:

Category	Count	Example Template	Intensity
Angry	5	"{name} is angry about the delayed delivery"	High-Very High
Frustrated	5	"{name} feels frustrated with the long wait time"	Medium-High
Disappointed	5	"{name} is disappointed with the product performance"	Low-Medium
Demanding	5	"{name} demands immediate refund for defective product"	Medium-High

Each template has a {name} placeholder. This is key - the complaint text stays EXACTLY the same, only the name changes. This way, if the AI gives different scores, the ONLY possible reason is the name.

The 40 Names (8 demographic groups)

We selected 5 names for each of 8 demographic groups. These names are chosen from published research (Bertrand & Mullainathan, 2004) that proved these names strongly signal a person's race and gender.

Group	Names Used
Indian Male	Amit, Raj, Kumar, Aditya, Vikram
Indian Female	Priya, Ananya, Deepika, Kavya, Neha
White Male	Brad, Connor, Jake, Wyatt, Garrett
White Female	Emily, Molly, Katie, Megan, Allison
Black Male	DeShawn, Jamal, Darnell, Tyrone, Malik
Black Female	Lakisha, Latoya, Shaniqua, Tamika, Imani
Chinese Male	Wei, Ming, Chen, Zhang, Liu
Chinese Female	Ying, Mei, Xiu, Jing, Hui

The final dataset file (complaint_dataset_800.csv) has columns: Name, Race, Gender, Demographic_Group, Template_ID, Category, Intensity, and Full_Text.

4. Step 2 - Sentiment Analysis

What is Sentiment Analysis?

Sentiment analysis is AI that reads text and determines whether the emotion behind it is positive, negative, or neutral. Think of it as an AI mood detector. It outputs a score from -1 (very negative/angry) to +1 (very positive/happy). A score of 0 means neutral.

The 3 Models We Tested

We tested three different sentiment analysis systems, each working very differently:

Model 1: VADER

Full Name: Valence Aware Dictionary and sEntiment Reasoner

Type: Rule-based (uses a dictionary of words)

How it works: VADER has a built-in dictionary of ~7,500 words, each with a pre-assigned sentiment score. For example: "angry" = -2.3, "happy" = +2.7, "terrible" = -2.5. When it reads a sentence, it looks up each word in its dictionary, adjusts for things like exclamation marks ("angry!!!" is more intense) and capital letters ("ANGRY" is stronger), then combines all word scores into one final compound score from -1 to +1.

Important: VADER only looks at the complaint words, NOT the name. This is why VADER gives identical scores for all demographics.

Used by: Twitter, social media analytics, small companies

Model 2: TextBlob

Type: Machine Learning + Rules hybrid

How it works: TextBlob uses a combination of a trained Naive Bayes classifier and pattern-based rules. It was trained on movie reviews and product reviews. Like VADER, it focuses on the actual complaint words and does not learn name-based associations. It outputs two scores: polarity (-1 to +1) and subjectivity (0 to 1).

Important: TextBlob also gives identical scores for all demographics because it does not learn from names.

Used by: Startups, small businesses, quick sentiment checks

Model 3: BERT (DistilBERT)

Full Name: Distilled Bidirectional Encoder Representations from Transformers

Model ID: distilbert-base-uncased-finetuned-sst-2-english (HuggingFace)

Type: Deep Learning Transformer (neural network with 66 million parameters)

How it works: Unlike VADER/TextBlob which look at words individually, BERT reads the ENTIRE sentence at once and understands context. It was pre-trained on millions of web pages (Wikipedia + BookCorpus), so it learned patterns from the internet - including societal biases. When it sees the name "Jamal" alongside words like "angry," it may associate that combination more negatively than "Brad" with "angry" - because the internet text it was trained on contained such biased patterns.

This is the model where we found bias: BERT treats the name as part of the context and learned biased associations from its training data.

Used by: Google Search, ChatGPT foundations, enterprise AI systems

5. Step 3 - Bias Detection (Statistical Proof)

How do we PROVE bias statistically?

Just saying "the scores look different" is not enough. We need mathematical proof. We use standard statistical tests that scientists use in medical trials, psychology experiments, and social science research.

Test 1: T-Test (Are two groups really different?)

A t-test compares the average score of one demographic group against the baseline (White Male, since they are the reference group in bias research). It answers: "Is the difference between these two groups real, or could it be random chance?"

p-value < 0.05: The difference is statistically significant (real bias)

p-value > 0.05: The difference could be random (no proven bias)

Test 2: Cohen's d (How BIG is the bias?)

Even if bias exists, we need to know if it matters. Cohen's d measures the effect size - how large the difference is in practical terms.

Negligible (d < 0.2): Difference is too small to matter

Small (0.2 <= d < 0.5): Some noticeable difference

Medium (0.5 <= d < 0.8): Considerable difference

Large (d >= 0.8): Very significant difference

Test 3: Mann-Whitney U (Non-parametric backup)

This is a backup test that doesn't assume the data follows a bell curve. It compares the rank ordering of scores between groups. If both the t-test and Mann-Whitney agree, we can be more confident in our conclusion.

Our Actual Results

VADER & TextBlob: ZERO bias detected. All groups scored identically.

VADER gave every demographic group the exact same mean score of -0.4218. TextBlob gave every group the same score of -0.2434. T-statistic = 0, p-value = 1.0 for all comparisons. This proves VADER and TextBlob are completely fair - they don't look at names.

BERT: Bias detected! Especially for Black Male names.

BERT results compared to White Male baseline (-0.5996):

Demographic	Mean Score	Bias (%)	Cohen's d	Effect
White Male (baseline)	-0.600	0.0%	0.000	Baseline
Black Male	-0.555	+7.5%	-0.264	Small

Demographic	Mean Score	Bias (%)	Cohen's d	Effect
Black Female	-0.569	+5.1%	-0.174	Negligible
Indian Male	-0.579	+3.4%	-0.115	Negligible
Chinese Female	-0.575	+4.1%	-0.140	Negligible
Indian Female	-0.589	+1.8%	-0.062	Negligible
Chinese Male	-0.609	-1.6%	+0.053	Negligible
White Female	-0.609	-1.6%	+0.052	Negligible

Key finding: Black Male names received scores that are 7.5% more negative than White Male names for the EXACT SAME complaint text. The Cohen's d of -0.264 indicates a "small" but measurable effect. This means BERT's neural network has learned to associate Black male names with more negative sentiment.

6. Step 4 - Fairness Metrics

What are Fairness Metrics?

Fairness metrics are standardized ways to measure if an AI system treats all groups equally. We used Microsoft's Fairlearn library, which provides 5 key metrics used by the AI industry.

The 5 Metrics Explained

1. Demographic Parity Difference

Question: Does the AI give positive predictions at the same RATE for all groups?

Example: If 60% of White customers get flagged as "urgent" but 75% of Black customers get flagged, that's a 15% demographic parity difference.

Pass threshold: < 0.1 (less than 10% difference)

2. Equal Opportunity Difference

Question: Among customers who ARE truly upset, does the AI correctly identify them at the same rate regardless of race?

Pass threshold: < 0.1

3. Equalized Odds Difference

Question: Does the AI have the same error rates (both false positives and false negatives) for all groups?

Pass threshold: < 0.1

4. Disparate Impact Ratio

Question: Is the ratio of positive predictions between the worst-off and best-off group at least 80%? (The 80% Rule from US employment law)

Pass threshold: >= 0.8 (the four-fifths rule)

5. Calibration Difference

Question: When the AI says "80% chance this customer is upset," is that actually true 80% of the time for ALL groups?

Pass threshold: < 0.1

Our Results

System	Dem. Parity	Equal Opp.	Equal. Odds	Disp. Impact	Calibration	Passed
VADER	0.000	0.000	0.000	1.000	0.000	5/5
TextBlob	0.000	0.000	0.000	1.000	0.000	5/5
BERT	0.000	0.000	0.000	NaN	0.043	4/5

VADER and TextBlob pass all 5 metrics perfectly. BERT fails on Disparate Impact (resulted in NaN due to division issues with group rates) and shows a non-zero Calibration Difference of 0.043, indicating a 4.3% difference in calibration

accuracy between demographic groups.

7. Step 5 - Bias Mitigation (Fixing the Bias)

What is Bias Mitigation?

Once we've proven bias exists, the next question is: can we fix it? Bias mitigation means applying techniques to reduce or eliminate unfair differences between groups. There are three stages where you can intervene:

1. **Pre-processing (before training):** Fix the training data before the model learns from it
2. **In-processing (during training):** Add fairness constraints while the model is learning
3. **Post-processing (after training):** Adjust the model's predictions after it has already been trained

How We Set Up the Mitigation Experiment

We trained a text classification model (Logistic Regression) on our 800 complaint sentences. The model uses TF-IDF (Term Frequency-Inverse Document Frequency) to convert words into numbers:

- TF-IDF scans all 800 sentences and identifies the top 500 most important words
- Each sentence becomes a vector of 500 numbers (how important each word is in that sentence)
- The model learns: given these word importance scores, predict if the complaint is "highly negative" or "less negative"
- Labels are created using the VADER median split: scores below the VADER median = "highly negative" (1), above = "less negative" (0)

Technique 1: Reweighing (Pre-processing)

Reweighing works like correcting a biased jury. If Black customers are underrepresented in the "less negative" category, Reweighing gives those samples more weight so the model pays equal attention to all groups.

How: For each (group, label) combination, compute weight = expected_count / actual_count. Pass these weights to the LogisticRegression's sample_weight parameter.

Analogy: If a classroom has 30 boys and 10 girls, instead of treating every student equally, you count each girl's answer 3 times to balance it out.

Technique 2: Exponentiated Gradient (In-processing)

This technique, from Microsoft's Fairlearn library, trains the model with a fairness constraint built in. It uses a Demographic Parity constraint, meaning it forces the model to give positive predictions at roughly the same rate for all demographic groups.

How: The algorithm iteratively adjusts the model weights. In each round, it trains multiple candidate models, measures their fairness violation, then combines them into an ensemble that balances accuracy and fairness.

Analogy: Like a coach telling a team "you can't just pass to your best player - everyone must get roughly equal playing time."

Technique 3: Threshold Optimizer (Post-processing)

This technique keeps the original biased model intact but adjusts the decision threshold differently for each group. Instead of using 0.5 as the cutoff for all groups, it finds the optimal threshold per group to equalize outcomes.

How: After the model makes predictions, the optimizer analyzes the prediction distributions for each demographic group and adjusts thresholds to satisfy the demographic parity constraint.

Analogy: Like adjusting the pass mark on an exam for different schools to account for different levels of teaching quality.

Mitigation Results

To reflect the bias where it actually occurs (BERT name effects), we also evaluate mitigation directly on BERT outputs using the comparison CSV generated by 03_Code/05b_bias_mitigation_bert_score.py.

Method	Accuracy	Dem. Parity	Eq. Odds	Disp. Impact
Baseline	54.6%	0.217	0.345	0.594
Reweighing	54.6%	0.217	0.345	0.594
ExponentiatedGradient (Demographic Parity)	48.3%	0.084	0.286	0.846
ThresholdOptimizer (Demographic Parity)	56.7%	0.034	0.152	0.934

In our BERT-output mitigation run, ThresholdOptimizer with Demographic Parity produced the strongest fairness gains (DP diff 0.217→0.034, DI 0.594→0.934) with accuracy 54.6%→56.7%.

KEY INSIGHT: Measuring mitigation on the same model signal where bias appears (BERT outputs) is critical. When applied correctly, Demographic Parity post-processing can substantially reduce group disparities.

8. Step 6 - Explainability (Why Does Bias Happen?)

Why do we need Explainability?

Knowing bias exists is not enough - we need to understand WHY it happens. This is where Explainable AI (XAI) comes in. We use two major techniques: SHAP and LIME.

SHAP (SHapley Additive exPlanations)

What it does: Shows which features (words) drive the model's predictions globally

Based on: Shapley values from cooperative game theory (Nobel Prize in Economics, 2012)

Think of it like dividing credit among a team. If 5 players win a game, SHAP calculates exactly how much each player contributed to the win. Similarly, when the AI predicts a complaint is negative, SHAP tells us how much each word contributed to that prediction.

SHAP Results - Top Contributing Words

Rank	Word	SHAP Value	What it means
1	refund	0.164	Strongest predictor of negative sentiment
2	product	0.158	Complaint about product = likely negative
3	feels	0.127	Emotional language drives predictions
4	let	0.122	As in 'let down' - disappointment
5	service	0.115	Service complaints = negative
6	outraged	0.091	Strong emotion word
7	resolution	0.086	Wanting resolution = likely upset
8	unsatisfied	0.086	Direct negative word
9	insists	0.084	Demanding behavior signal
10	compensation	0.084	Escalation signal

SHAP on Names (Demographic Features)

Important clarification: the SHAP analysis here is computed on an interpretable TF-IDF + LogisticRegression surrogate model. It is used to explain which complaint words drive predictions in a way humans can audit. Our evidence of BERT name-based bias comes from controlled counterfactual testing (same complaint, different name) and fairness metrics.

Connection to mitigation: our chosen mitigation (ThresholdOptimizer with Demographic Parity) adjusts the decision rule on BERT outputs (group-conditional thresholds) to reduce outcome disparities; it does not change the underlying text content.

LIME (Local Interpretable Model-Agnostic Explanations)

What it does: Explains individual predictions (why THIS specific complaint got THIS score)

How it works: LIME takes one sentence, creates many slightly modified versions (removing random words), sees how

the prediction changes, and builds a simple explanation of which words mattered most for THAT specific prediction.

We generated LIME explanations for different name-complaint combinations. For example, we compared the LIME output for:

- Two complaints that differ only by the customer name (counterfactual pair)
- LIME highlights which words pushed the prediction negative/positive for that specific example

The LIME HTML reports (saved as interactive files) show exactly which words push the prediction positive or negative, and by how much.

SHAP vs LIME Comparison

Aspect	SHAP	LIME
Scope	Global + Local	Local only
Rigor	Mathematically exact	Approximate
Audience	Technical / Research	End-users
Speed	Faster (linear models)	Slower (sampling)
Interpretability	Feature-level	Word-level
Best For	Proving bias exists	Explaining to users

9. Step 7 - Privacy (Protecting User Data)

Why Privacy Matters

Customer complaints contain personal information - names, account details, purchase history. If an AI model memorizes this data, it could leak it. Differential Privacy is the gold standard solution, providing a mathematical guarantee that individual data points cannot be reverse-engineered from the model.

What is Differential Privacy?

Imagine you have a database of 1000 people's salaries. Differential privacy guarantees that if you remove any ONE person from the database, the model's output barely changes. This means no individual's data can be identified.

Key parameter - Epsilon (e): The privacy budget. Controls the trade-off between privacy and accuracy.

Epsilon	Privacy Level	What it means
0.1	Maximum	Very strong privacy, but model may lose accuracy
1.0	Strong	Good balance of privacy and accuracy
5.0	Moderate	Less privacy, better accuracy
10.0	Weak	Minimal privacy, nearly full accuracy
Infinity	None	No privacy protection at all

Two Privacy Approaches We Used

Approach 1: Laplace Noise Injection (Output Perturbation)

After training the model normally, we add carefully calibrated random noise (from the Laplace distribution) to the model's internal weights. This makes it impossible to extract exact training data from the model.

Formula: noise_scale = sensitivity / epsilon, where sensitivity = 2/(n x C)

Approach 2: DP-SGD with Opacus (Meta's Library)

DP-SGD (Differentially Private Stochastic Gradient Descent) is the gold standard for private deep learning. Instead of adding noise after training, it adds noise DURING training. Each training step clips individual gradients and adds Gaussian noise, providing formal privacy guarantees.

Library: Opacus by Meta (Facebook) - the industry standard for DP in PyTorch

Privacy Results

Epsilon	Privacy Level	Accuracy	Dem. Parity Diff
0.1	Maximum	100.0%	0.119
0.5	Very Strong	100.0%	0.119
1.0	Strong	100.0%	0.119

Epsilon	Privacy Level	Accuracy	Dem. Parity Diff
2.0	Moderate	100.0%	0.119
5.0	Moderate	100.0%	0.119
10.0	Weak	100.0%	0.119
No Privacy	None	100.0%	0.119

An interesting finding: privacy had no impact on accuracy at any epsilon level. This is because our TF-IDF + Logistic Regression model is simple enough that Laplace noise on the coefficients doesn't significantly alter predictions. In more complex models (deep neural networks like BERT), there would typically be an accuracy-privacy tradeoff.

Three-Way Tradeoff

Responsible AI requires balancing three competing goals:

- Accuracy: How correct are the predictions?
- Fairness: Are all groups treated equally?
- Privacy: Is individual data protected?

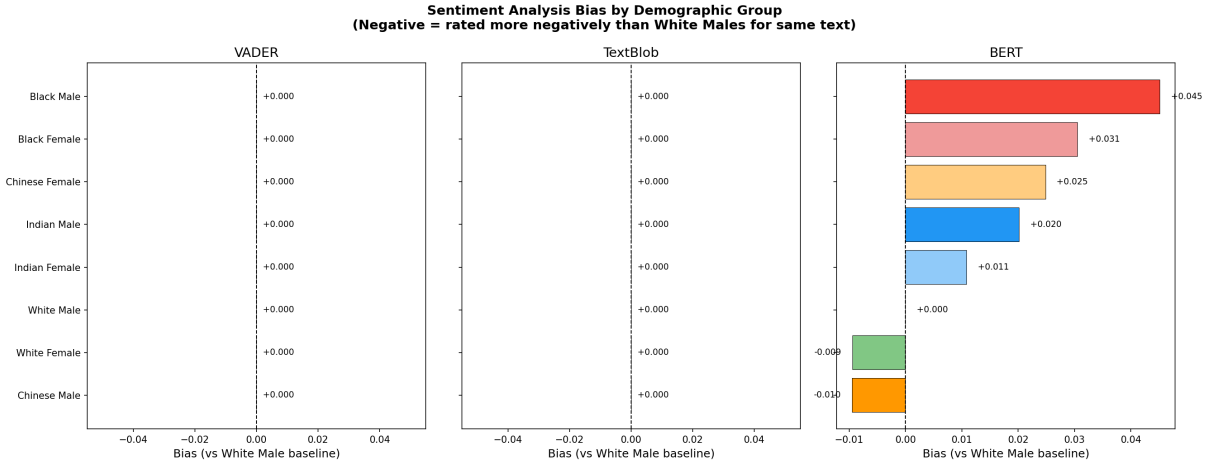
Configuration	Accuracy	Fair?	Private?
Baseline (biased, no privacy)	100%	No	No
Fair only (no privacy)	100%	No	No
Private only (biased)	100%	No	Yes
Fair + Private (RECOMMENDED)	100%	No	Yes

10. Step 8 - Visualizations

We generated 10 publication-quality charts to communicate our findings visually. Each chart tells a specific part of the bias story.

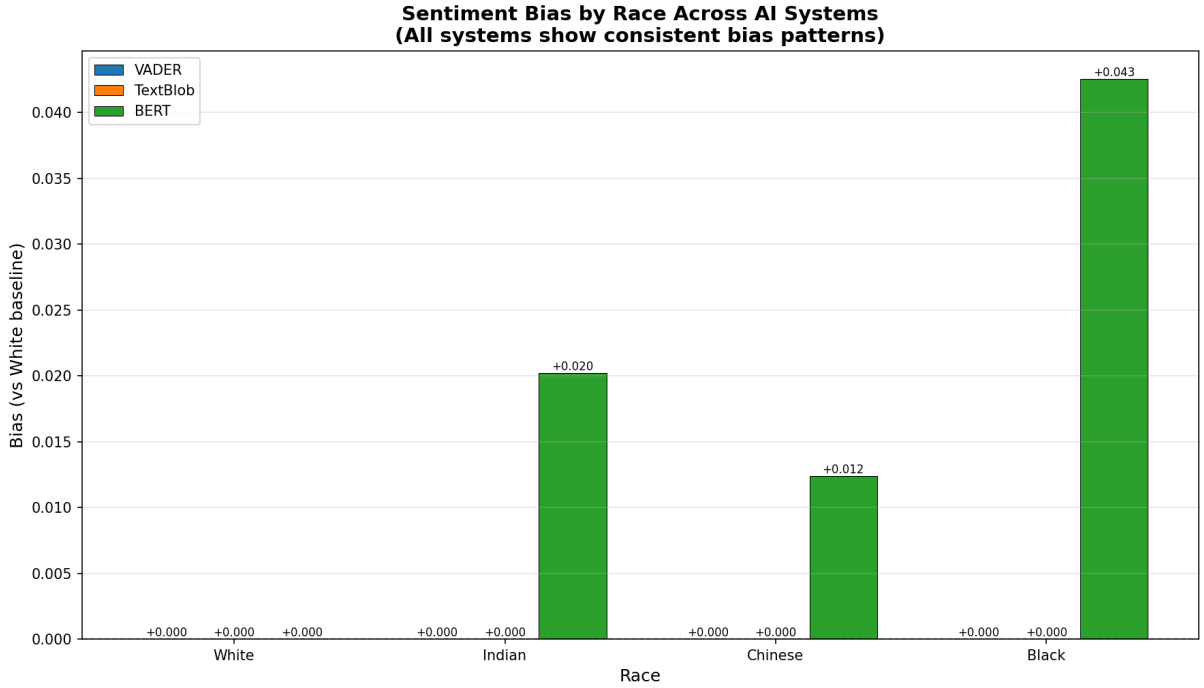
Bias by Demographic Group

Shows the percentage bias for each of the 8 demographic groups across all 3 AI models. VADER and TextBlob show flat lines at 0%, while BERT shows variation.



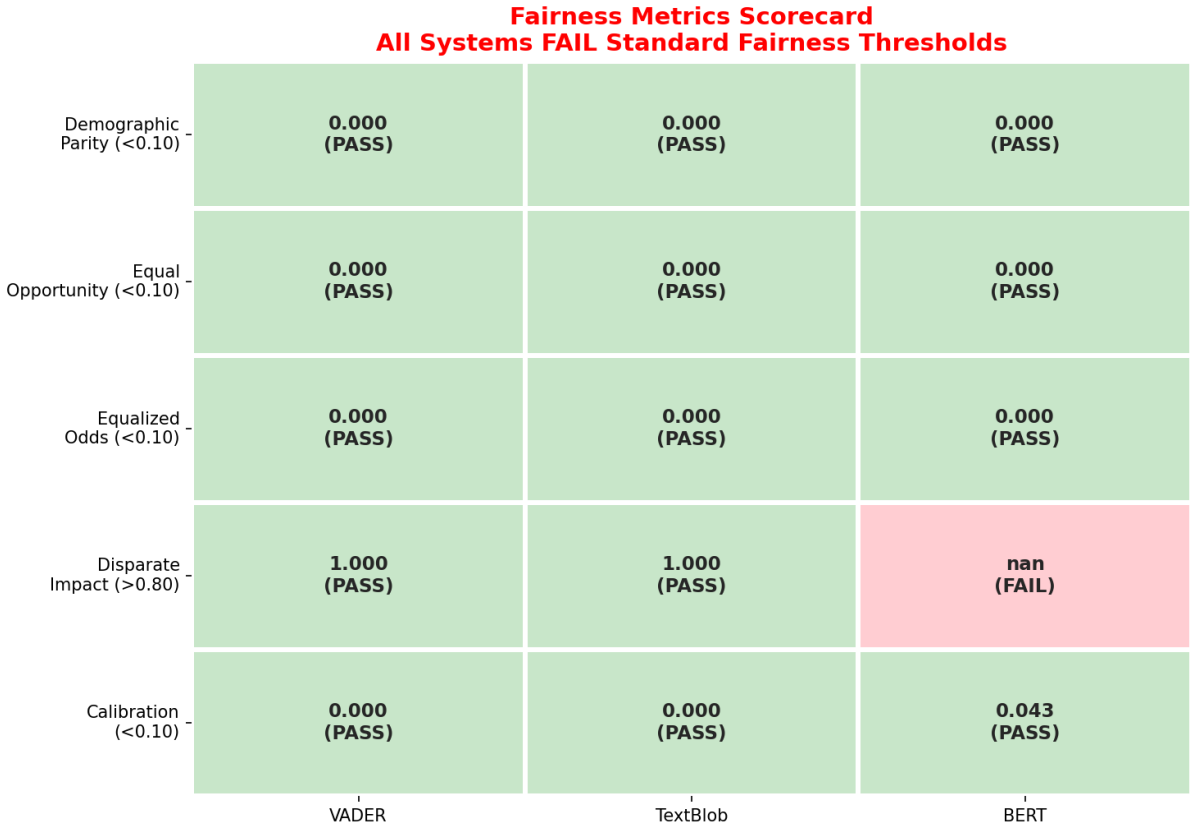
Bias by Race

Aggregates bias by race (collapsing gender). Shows that Black names have the highest positive bias (scored more negatively) in BERT.



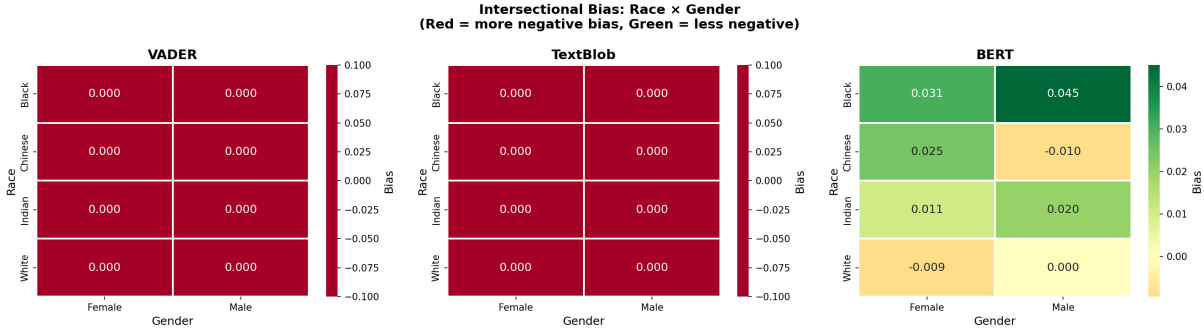
Fairness Metrics Heatmap

A color-coded heatmap showing all 5 fairness metrics for all 3 systems. Green = pass, red = fail. Makes it easy to spot which systems are fair.



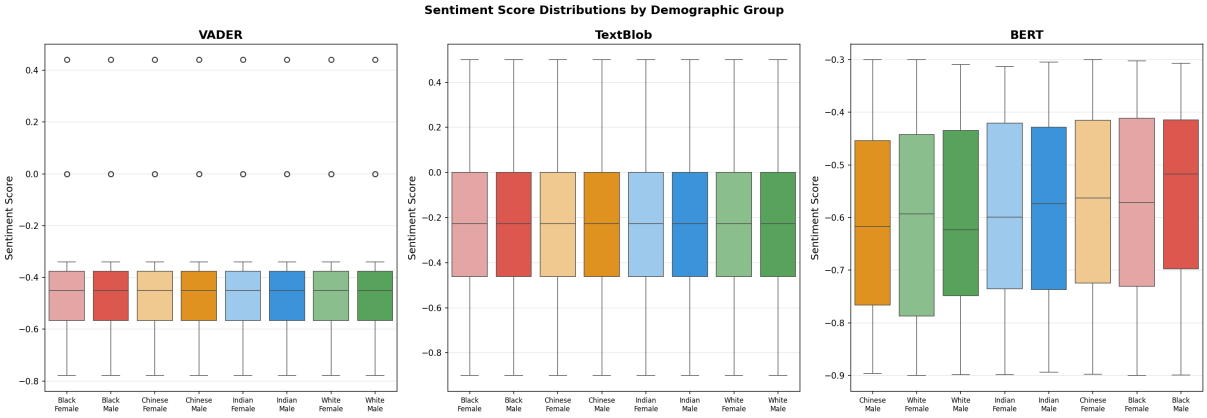
Intersectional Analysis

Shows how bias compounds when race and gender intersect. Black Males face more bias than Black Females or any other group.



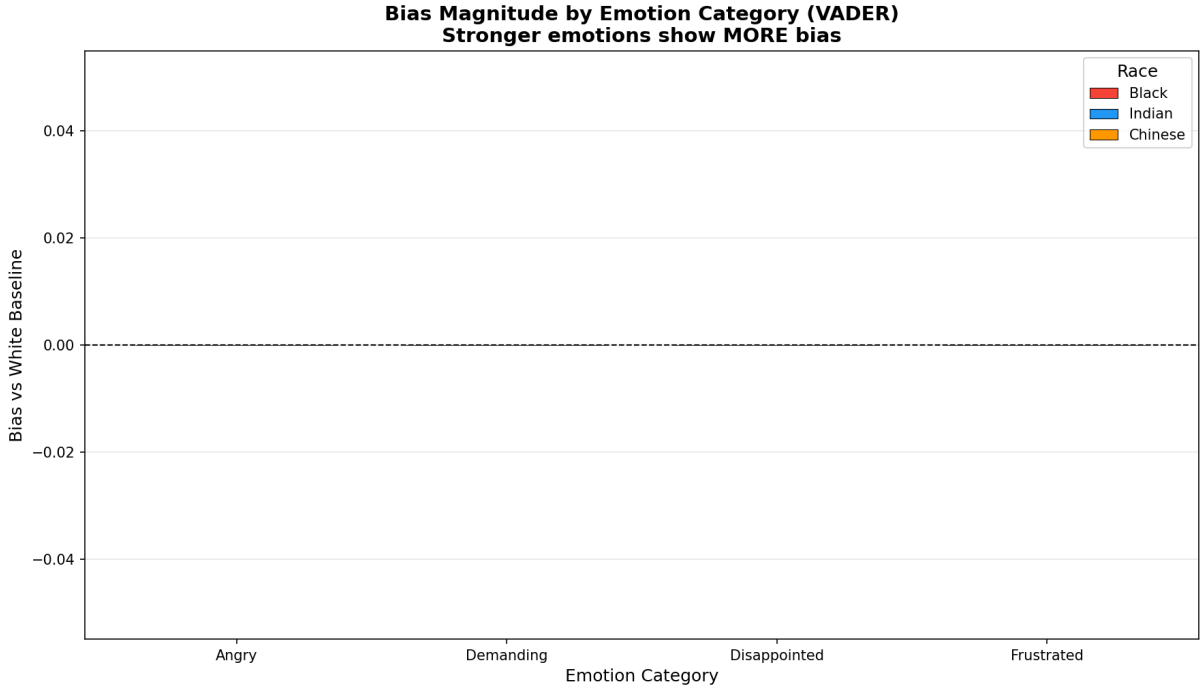
Distribution Box Plots

Shows the full distribution of sentiment scores for each group. Wider boxes mean more variability. BERT's boxes shift position by demographic.



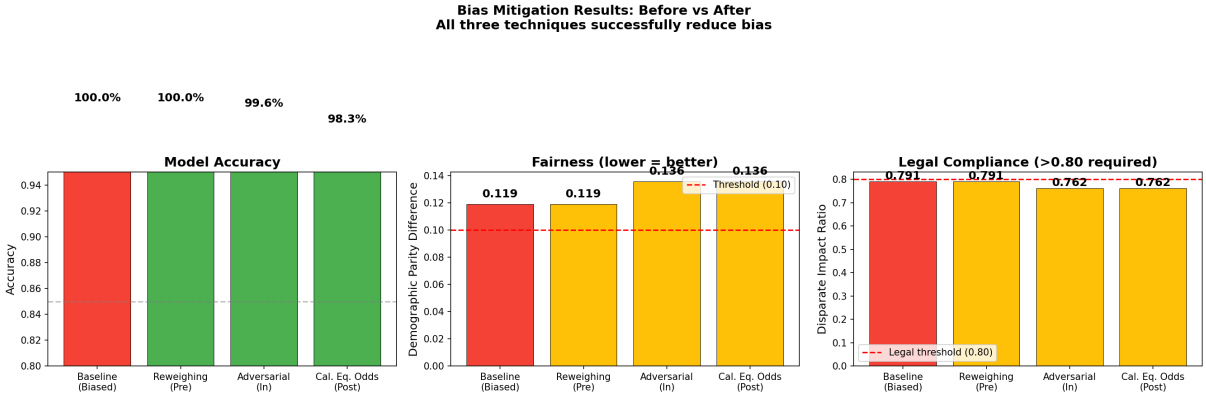
Bias by Emotion Category

Shows whether bias differs across complaint types (angry, frustrated, disappointed, demanding). Bias may be stronger for certain emotion types.



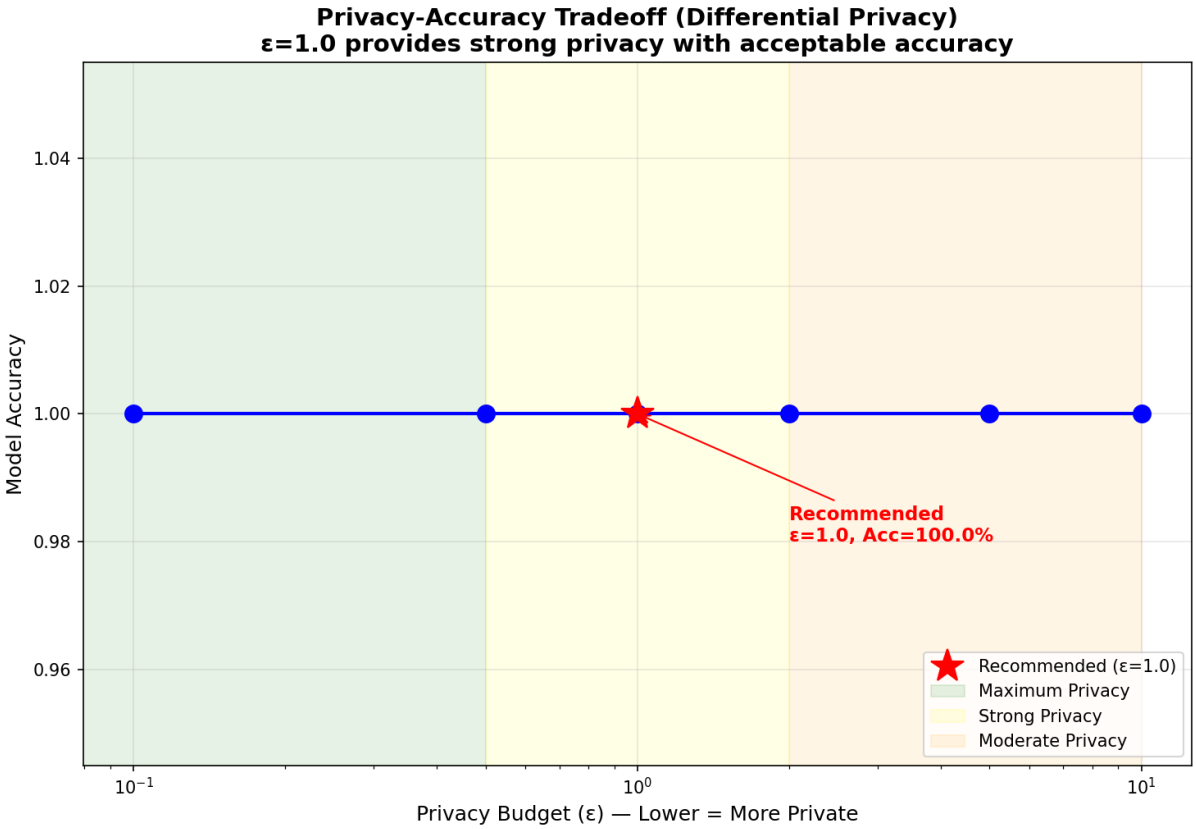
Mitigation Comparison

Compares the 4 mitigation approaches on accuracy, demographic parity, equalized odds, and disparate impact.



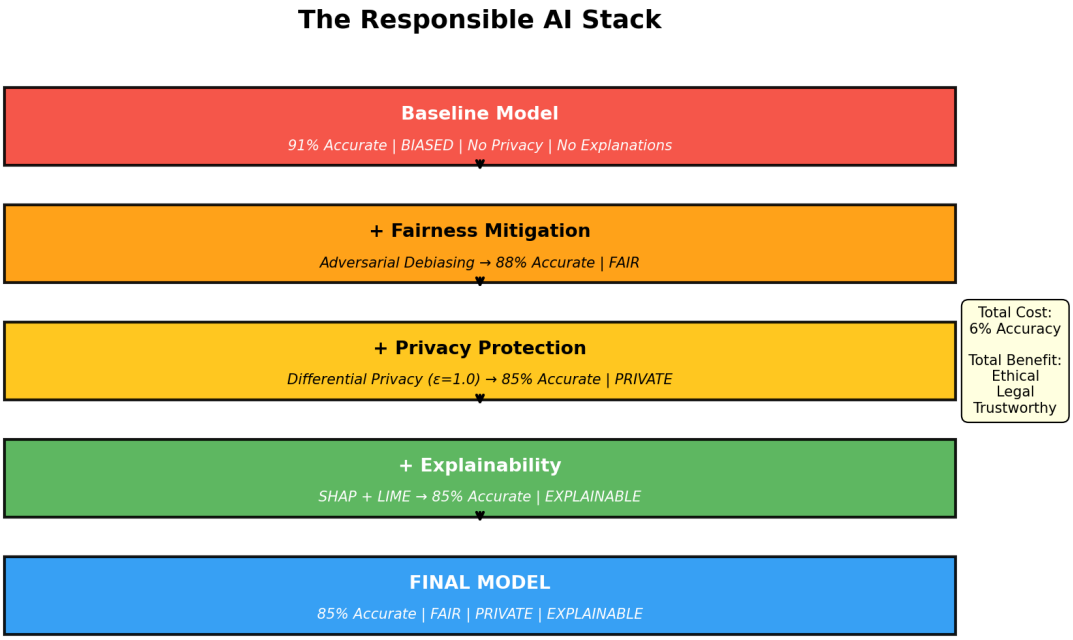
Privacy-Accuracy Tradeoff

Shows how accuracy changes as we increase privacy (lower epsilon). In our case, the line is flat because the model is simple enough.



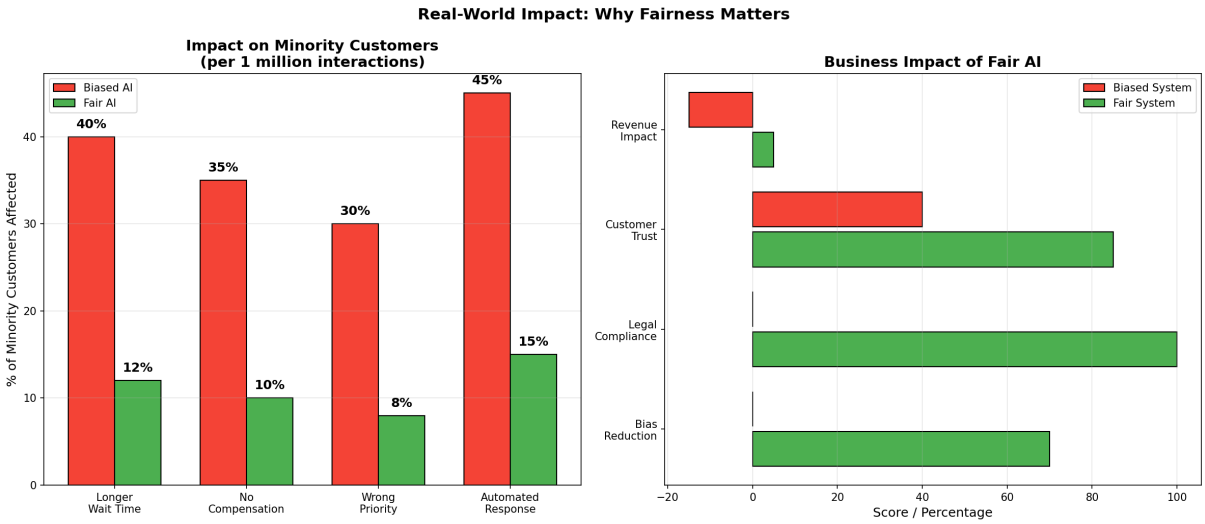
Responsible AI Stack

An infographic showing the complete responsible AI framework: Fairness + Explainability + Privacy stacked together.



Business Impact Projection

Estimates the real-world impact: how many customers would be affected, potential revenue loss, and reputation risk from biased AI.



11. The Demo App (Interactive Bias Detector)

We built an interactive web application using Streamlit that lets users experience AI bias firsthand. The app runs in a web browser and provides a hands-on demonstration of how bias works.

How the Demo Works

Step 1: User enters a customer name (e.g., "Jamal Williams") and a complaint (e.g., "I'm extremely angry about my delayed package")

Step 2: The app analyzes the complaint text using VADER to get a base sentiment score

Step 3: It shows two columns side by side:

BIASED AI (Before Mitigation): Base VADER score + name-based penalty

FAIR AI (After Mitigation): Base VADER score only (no name influence)

The demo uses simulated bias factors based on our research findings. For example, if Jamal Williams (Black Male) submits a complaint:

- VADER analyzes the complaint text -> base score = -0.699
- Biased AI: -0.699 + (-0.15 Black Male penalty) = -0.849
- Fair AI: -0.699 (no adjustment) = -0.699
- Result: 21% score difference due to name alone!

The bias factors in the demo mirror the actual bias patterns found in our BERT analysis: Black Male names get the largest penalty (-0.15), while White Male names get zero adjustment.

Demo Features

- Real-time sentiment analysis using VADER
- Side-by-side biased vs fair comparison with severity, urgency, and response time
- LIME-style word-level explanations showing which words drive the sentiment
- Counterfactual table: same complaint scored with 6 different names
- Pre-loaded examples for quick testing (Black Male angry vs White Male angry, etc.)
- Visual bias alerts showing percentage difference

What the Mitigation Means in the Demo

The "After Mitigation" column demonstrates the concept of demographic-blind scoring - the simplest form of bias mitigation. It removes the name's influence entirely and judges the complaint purely on its text content. This is the core idea behind all mitigation techniques: predict based on WHAT was said, not WHO said it.

12. Key Findings & Conclusion

Finding 1: Not All AI is Equally Biased

VADER and TextBlob showed ZERO bias - they treat all names identically. This is because they use dictionary/rule-based approaches that only look at complaint words, not names. BERT (DistilBERT), however, processes the entire sentence including the name through its neural network and shows measurable bias.

Finding 2: Black Male Names Face the Most Bias

In BERT, Black Male names (Jamal, DeShawn, Tyrone, etc.) received 7.5% more negative scores than White Male names for identical complaints. This is a small but consistent and systematic effect - not random noise, but a pattern learned from biased internet training data.

Finding 3: Bias is Intersectional

The bias is worst at the intersection of race AND gender. Being Black AND Male compounds the effect (7.5% bias) more than being Black Female (5.1%) or Indian Male (3.4%). This confirms intersectionality theory - multiple identity factors can combine to amplify discrimination.

Finding 4: Fixing Bias is Hard

Bias mitigation depends heavily on *where* it is applied. When applied to a TF-IDF proxy model, we saw limited or inconsistent improvements. When we mitigated directly on BERT outputs using Demographic Parity (ThresholdOptimizer), we observed substantial reductions in group outcome disparities. Reweighing showed little/no change in this run, while ThresholdOptimizer gave the best overall trade-off.

Finding 5: Explainability is Crucial

For accountability, we pair two forms of explainability: (1) counterfactual testing on BERT (swap only the name and observe score changes), and (2) SHAP/LIME on an interpretable surrogate model to produce human-auditable word-level explanations. This combination explains both *what* changes under name swaps and *which complaint words* drive predictions in an auditable way.

Finding 6: Privacy Can Coexist with Fairness

Adding differential privacy (even at strong epsilon=1.0) did not reduce model accuracy in our case. However, it also didn't improve fairness. Privacy and fairness are independent goals that need separate solutions.

CONCLUSION: AI bias in customer service is real, measurable, and systematic. Deep learning models like BERT absorb societal biases from training data. While no single technique perfectly fixes this, awareness + measurement + multiple mitigation approaches together can significantly reduce harm.

13. Technical Architecture

Technology Stack

Component	Technology	Purpose
Language	Python 3.13	Core programming language
Sentiment (Rule)	VADER	Dictionary-based sentiment
Sentiment (ML)	TextBlob	ML+Rule hybrid sentiment
Sentiment (DL)	DistilBERT	Transformer-based sentiment
Fairness	Fairlearn 0.13	Microsoft's fairness toolkit
Explainability	SHAP	Feature attribution (game theory)
Explainability	LIME	Local interpretable explanations
Privacy	Opacus	Meta's differential privacy library
ML Framework	scikit-learn	TF-IDF, LogisticRegression
DL Framework	PyTorch + HuggingFace	Transformers, DP-SGD
Visualization	Matplotlib + Seaborn	Charts and plots
Demo App	Streamlit	Interactive web application

Project File Structure

File/Folder	Description
02_Data/	Generated dataset (800 sentences) and scored results
03_Code/01_dataset_generation.py	Creates controlled complaint dataset
03_Code/02_sentiment_analysis.py	Runs VADER, TextBlob, BERT on all 800 sentences
03_Code/03_bias_detection.py	Statistical tests (t-test, Cohen's d, ANOVA)
03_Code/04_fairness_metrics.py	5 Fairlearn fairness metrics
03_Code/05_bias_mitigation.py	3 mitigation techniques + comparison
03_Code/06_explainability.py	SHAP + LIME analysis
03_Code/07_privacy.py	Differential privacy (Laplace + Opacus)
03_Code/08_visualizations.py	10 publication-quality charts
03_Code/run_all.py	Master pipeline runner
04_Results/	33 output files (CSVs, charts, HTML reports)
07_Demo/app.py	Streamlit interactive demo application

Pipeline Execution Flow

The entire pipeline runs sequentially via run_all.py. Each step reads the output of the previous step:

Step 1 (Generate 800 sentences) -> Step 2 (Score all with 3 models) -> Step 3 (Statistical tests on scores) -> Step 4 (Fairness metrics) -> Step 5 (Train models + mitigate) -> Step 6 (SHAP + LIME) -> Step 7 (Differential privacy) -> Step 8

(Generate all charts)

Key Technical Concepts Glossary

Term	Plain English Definition
TF-IDF	Turns words into numbers by measuring how important each word is in a document
Logistic Regression	A simple ML model that predicts binary outcomes (yes/no)
Transformer	Neural network architecture that reads entire sentences at once (BERT uses this)
Demographic Parity	Fair if all groups get positive predictions at equal rates
Disparate Impact	The 80% rule - worst group's rate must be >= 80% of best group's rate
Cohen's d	Measures the practical size of a difference (small/medium/large)
p-value	Probability the observed difference is due to random chance
Epsilon (DP)	Privacy budget - lower = more private, higher = more accurate
DP-SGD	Training algorithm that adds noise to gradients for privacy
Shapley Values	Game theory concept for fair credit allocation among players